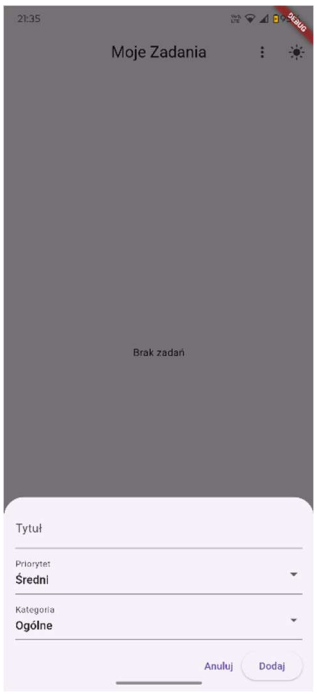
Sprawozdanie

Aplikacja TODO z Firebase

Wiktor Zmiendak

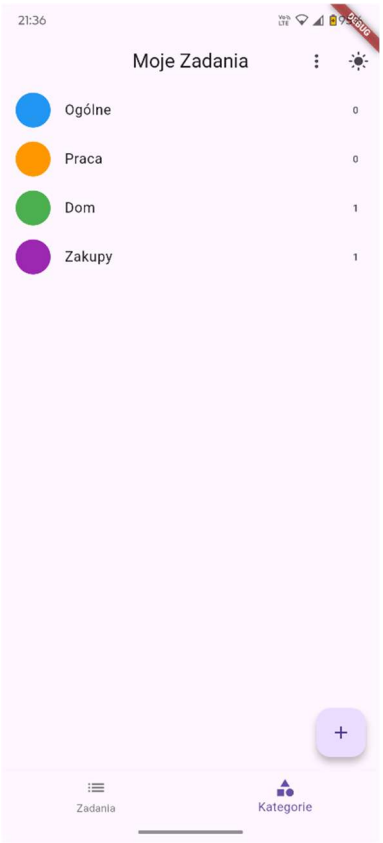
1. **Opis aplikacji:**
   Aplikacja służy do wyświetlania i zapisywania zadań w postaci przejrzystego notatnika. Dzięki swoim funkcjonalnością pokazuje obecne zadania, pozwala szybko dodać nowe, edytować lub oznaczyć jako wykonane. Po uruchomieniu próbuje nawiązać połączenie z Firebase a następnie na bieżąco prowadzi synchronizacje zmian.
2. **Interface:**
   Interfejs jest zaprojektowany tak, żeby wykonywanie operacji było jak najkrótsze, oto parę przykładów:
   - Aby dodać nowe zadanie należy dotknąć plusa, wpisać tytuł (plus parę dodatkowych opcji) i zatwierdzić



- W celu usunięcia zadania wystarczy przeciągnąć element a następnie potwierdzić swoją operację



- Jeśli chcemy zobaczyć dostępne kategorie i liczbę zadań w każdej z nich wystarczy zmienić sekcję na „Kategorie” w dolnym pasku



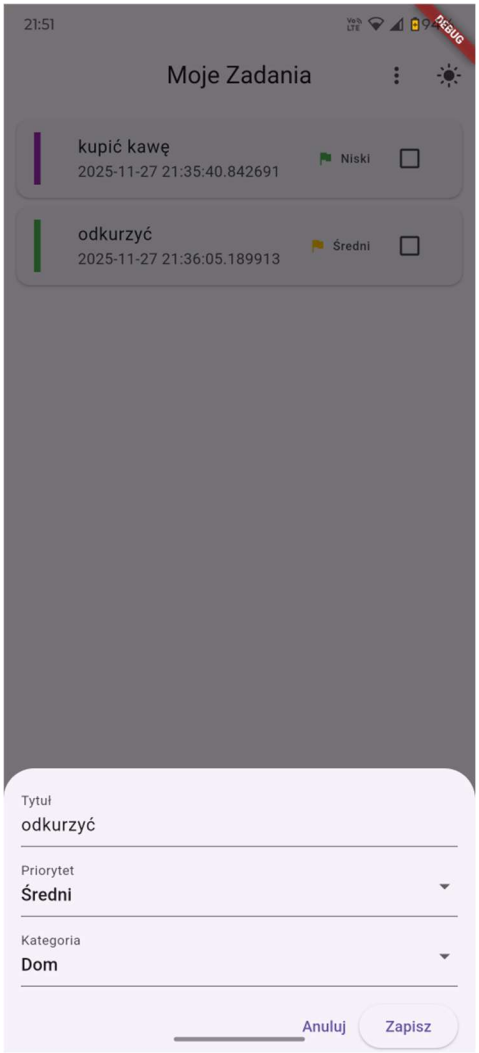
3. **Najważniejsze funkcje:**
    - Dodawanie zadania - funkcja _showAddDialog() otwiera formularz w dolnym arkuszu i po jego walidacji tworzy dokument w tasks. Po poprawnym zapisie modal się zamyka.

```
todo_app_zmiendak_wiktor > lib > main.dart > _TodoHomeState
73  class _TodoHomeState extends State<TodoHome> {
120   Future<void> _showAddDialog() async {
145                 (v == null || v.trim().isEmpty) ? 'Wpisz tytuł' : null,
146               ), // TextFormField
147               const SizedBox(height: 8),
148               DropdownButtonFormField<Priority>(
149                 value: selectedPriority,
150                 decoration: const InputDecoration(labelText: 'Priorytet'),
151                 items: Priority.values
152                     .map(
153                       (p) => DropdownMenuItem(
154                         value: p,
155                         child: Text(_priorityText(p)),
156                       ), // DropdownMenuItem
157                     )
158                     .toList(),
159                 onChanged: (v) => selectedPriority = v ?? Priority.medium,
160               ), // DropdownButtonFormField
161               const SizedBox(height: 8),
162               DropdownButtonFormField<String>(
163                 value: selectedCategory,
164                 decoration: const InputDecoration(labelText: 'Kategoria'),
165                 items: _categoryColors.keys
166                     .map((k) => DropdownMenuItem(value: k, child: Text(k)))
167                     .toList(),
168                 onChanged: (v) =>
169                     selectedCategory = v ?? _categoryColors.keys.first,
170               ), // DropdownButtonFormField
171               const SizedBox(height: 12),
172               Row(
173                 mainAxisAlignment: MainAxisAlignment.end,
174                 children: [
175                   TextButton(
176                     onPressed: () => Navigator.of(c).pop(),
177                     child: const Text('Anuluj'),
178                   ), // TextButton
179                   ElevatedButton(
180                     onPressed: () async {
181                       if (!(key.currentState?.validate() ?? false)) return;
182                       final title = _titleController.text.trim();
183                       await tasksRef.add({
184                         'title': title,
185                         'createdAt': FieldValue.serverTimestamp(),
186                         'date': Timestamp.fromDate(DateTime.now()),
187                         'completed': false,
188                         'priority': selectedPriority.index,
189                         'category': selectedCategory,
190                       });
191                       if (mounted) Navigator.of(c).pop();
192                     },
193                     child: const Text('Dodaj'),
194                   ), // ElevatedButton
195                 ],
196               ), // Row
197               const SizedBox(height: 12),
198             ],
199           ), // Column
200         ), // Form
```



- Edycja zadania – funkcja _showEditDialog() działa bardzo podobnie do dodawania tzn. wypełnia pola aktualnymi danymi, pozwala zmienić tytuł, priorytet i kategorię, a następnie aktualizuje dokument w Firestore.

```
73  class _TodoHomeState extends State<TodoHome> {
205   Future<void> _showEditDialog(DocumentSnapshot doc) async {
225           key: key,
226           child: Column(
227             mainAxisSize: MainAxisSize.min,
228             children: [
229               TextFormField(
230                 controller: editController,
231                 decoration: const InputDecoration(labelText: 'Tytuł'),
232                 validator: (v) =>
233                     (v == null || v.trim().isEmpty) ? 'Wpisz tytuł' : null,
234               ), // TextFormField
235               const SizedBox(height: 8),
236               DropdownButtonFormField<Priority>(
237                 value: selectedPriority,
238                 decoration: const InputDecoration(labelText: 'Priorytet'),
239                 items: Priority.values
240                     .map(
241                       (p) => DropdownMenuItem(
242                         value: p,
243                         child: Text(_priorityText(p)),
244                       ), // DropdownMenuItem
245                     )
246                     .toList(),
247                 onChanged: (v) => selectedPriority = v ?? Priority.medium,
248               ), // DropdownButtonFormField
249               const SizedBox(height: 8),
250               DropdownButtonFormField<String>(
251                 value: selectedCategory,
252                 decoration: const InputDecoration(labelText: 'Kategoria'),
253                 items: _categoryColors.keys
254                     .map((k) => DropdownMenuItem(value: k, child: Text(k)))
255                     .toList(),
256                 onChanged: (v) =>
257                     selectedCategory = v ?? _categoryColors.keys.first,
258               ), // DropdownButtonFormField
259               const SizedBox(height: 12),
260               Row(
261                 mainAxisAlignment: MainAxisAlignment.end,
262                 children: [
263                   TextButton(
264                     onPressed: () => Navigator.of(c).pop(),
265                     child: const Text('Anuluj'),
266                   ), // TextButton
267                   ElevatedButton(
268                     onPressed: () async {
269                       if (!(key.currentState?.validate() ?? false)) return;
270                       final title = editController.text.trim();
271                       await doc.reference.update({
272                         'title': title,
273                         'priority': selectedPriority.index,
274                         'category': selectedCategory,
275                       });
276                       if (mounted) Navigator.of(c).pop();
277                     },
278                     child: const Text('Zapisz'),
279                   ), // ElevatedButton
```

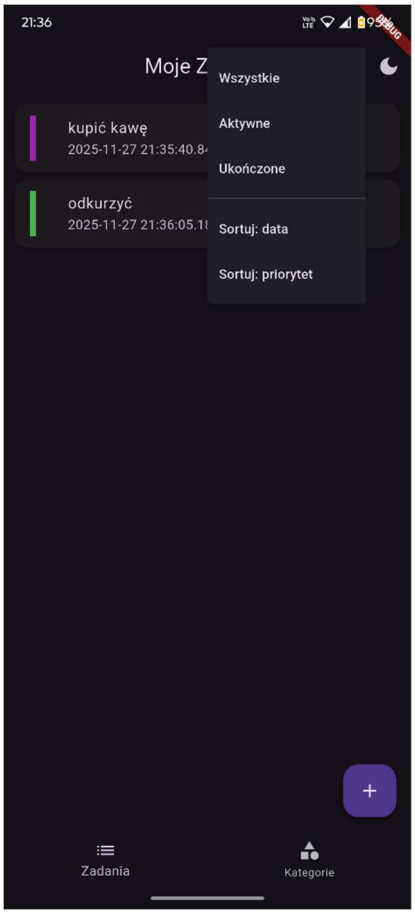


- Filtracja i sortowanie – funkcja _applyFilterAndSort() po pobraniu bieżących dokumentów filtruje je według trybu wszystkie/aktywne/ukończone i sortuje je po dacie albo po priorytecie. Dzięki temu można w prosty sposób zorientować się w pilnych rzeczach.

```dart
List<QueryDocumentSnapshot> _applyFilterAndSort(
  List<QueryDocumentSnapshot> docs,
) {
  var list = docs.toList();
  list = list.where((d) {
    final data = d.data() as Map<String, dynamic>;
    final completed = data['completed'] ?? false;
    switch (_filter) {
      case FilterMode.all:
        return true;
      case FilterMode.active:
        return completed == false;
      case FilterMode.completed:
        return completed == true;
    }
  }).toList();

  if (_sort == SortMode.date) {
    list.sort((a, b) {
      final ma = (a.data() as Map<String, dynamic>)['date'] as Timestamp?;
      final mb = (b.data() as Map<String, dynamic>)['date'] as Timestamp?;
      final da = ma?.toDate() ?? DateTime.fromMillisecondsSinceEpoch(0);
      final db = mb?.toDate() ?? DateTime.fromMillisecondsSinceEpoch(0);
      return da.compareTo(db);
    });
  } else if (_sort == SortMode.priority) {
    list.sort((a, b) {
      final pa =
          (a.data() as Map<String, dynamic>)['priority'] as int? ??
          Priority.medium.index;
      final pb =
          (b.data() as Map<String, dynamic>)['priority'] as int? ??
          Priority.medium.index;
      return pb.compareTo(pa);
    });
  }

  return list;
}
```

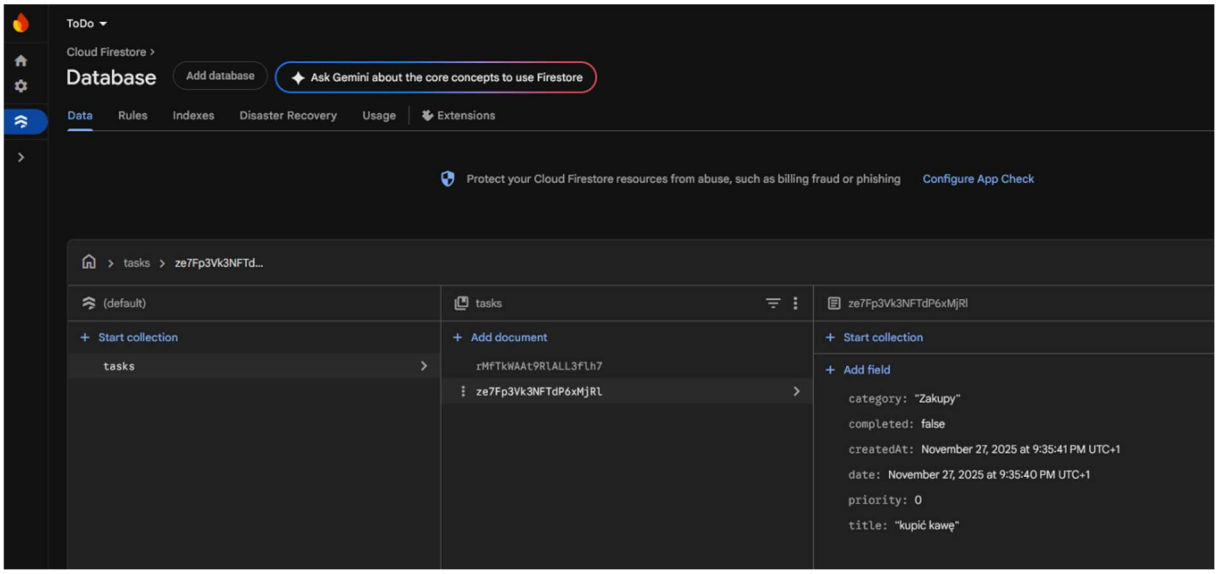


- Kategorie i kolorowanie - _categoryColors odpowiada za dobieranie koloru dla danej kategorii, _buildCategoriesView zlicza nam i wyświetla dostępne kategorie

```dart
Widget _buildCategoriesView(List<QueryDocumentSnapshot> docs) {
  final counts = <String, int>{};
  for (final d in docs) {
    final cat =
        (d.data() as Map<String, dynamic>)['category'] as String? ?? 'Ogólne';
    counts[cat] = (counts[cat] ?? 0) + 1;
  }
  final keys = _categoryColors.keys.toList();
  return ListView.builder(
    itemCount: keys.length,
    itemBuilder: (c, i) {
      final k = keys[i];
      return ListTile(
        leading: CircleAvatar(backgroundColor: _categoryColors[k]),
        title: Text(k),
        trailing: Text('${counts[k] ?? 0}'),
      ); // ListTile
    },
  ); // ListView.builder
}
```

```dart
final Map<String, Color> _categoryColors = {
  'Ogólne': Colors.blue,
  'Praca': Colors.orange,
  'Dom': Colors.green,
  'Zakupy': Colors.purple,
};
```

- Synchronizacja z Firebase - Aplikacja używa Firebase Auth (anonimowego logowania) i Cloud Firestore. Po inicjalizacji SDK aplikacja subskrybuje kolekcję tasks jako strumień a Firestore przesyła w czasie rzeczywistym aktualizacje dokumentów do klientów. Zapisy w aplikacji wysyłają zmiany na serwer a następnie wszyscy podłączeni klienci otrzymają aktualizacje automatycznie.




```
Run | Debug | Profile
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  try {
    await FirebaseAuth.instance.signInAnonymously();
  } catch (e, st) {
    debugPrint('Anonymous sign-in failed: $e');
    debugPrint('$st');
  }
  final prefs = await SharedPreferences.getInstance();
  final isDark = prefs.getBool('darkMode') ?? false;
  runApp(MyApp(initialDarkMode: isDark));
}
```

```
    body: StreamBuilder<QuerySnapshot>(
      stream: tasksRef.snapshots(),
      builder: (context, snap) {
        if (snap.hasError) return Center(child: Text('Błąd: ${snap.error}'));
        if (snap.connectionState == ConnectionState.waiting)
          return const Center(child: CircularProgressIndicator());
        final docs = snap.data?.docs ?? [];
        if (_navIndex == 0) return _buildTasksView(docs);
        return _buildCategoriesView(docs);
      },
    ), // StreamBuilder
```

- Z dodatkowych mniejszych elementów wpływających na jakość z korzystania można wymienić:
    - Dark Mode – w prawym górnym rogu istnieje możliwość zmiany theme na jasne/ciemne
    - Priorytetyzacja zadań – podczas tworzenia zadań można im nadać wagę niski/średni/wysoki
    - Odznaczanie wykonanych zadań przy pomocy checkboxów przy zadaniach